

UI Event-action table의 구현과 Region 실행 구조 설명

2026-09-15 작업 폴더의 실제 소스를 기준으로 작성했습니다. [UI_Event_Action_Table.md](#)의 **32개 표, 182개 ID 전체**를 조사하고, [기존 TradingSTM 구현 설명 문서](#)의 발표식 설명·구현 위치·실제 코드 예시·전체 ID 색인 구성을 따랐습니다. 관련 UI 테스트 15개 파일, 113개 테스트가 통과했습니다. 이 작업에서는 설명 문서만 추가했습니다.

교수님, UI 설계의 핵심은 **상단 상태, 화면 이동, 차트 설정, 거래 내역, 종료 절차**가 각각 자기 상태를 유지하면서 이벤트에 반응하게 하는 것입니다. 현재 구현은 React로 화면을 그리고 XState로 상태 전이를 관리합니다. 다만 설계의 거대한 STM 계층을 그대로 하나의 객체로 옮기지는 않았습니다. **UiApplicationFacade**가 **12개의 독립적인 XState actor**를 보관하고, 차트와 CSV 설정처럼 실제 직교 상태가 필요한 부분에는 **type: 'parallel'**을 사용합니다.

따라서 “Region을 나누어 독립적인 상태를 유지하려는 의도는 상당 부분 구현되었다”고 설명할 수 있습니다. 다만 **모든 Region**이 설계와 같은 계층의 병렬 상태로 구현된 것은 아니며, 모든 표의 동작이 원문과 완전히 같지도 않습니다. 정지 중 REGIME 선택, 필터 상태 표현, 화면 복귀의 history 범위 등은 별도로 구분해야 합니다.

1. 설계 문서와 실제 구현 폴더의 대응

교수님, **Design/UI**는 설계 자료를 보관하는 폴더이고 실제 UI 구현은 **UI/src** 아래에 있습니다. 폴더마다 다음 책임을 맡겼습니다.

폴더	책임	읽기 시작할 파일
UI/src/app	앱 구성, 기능별 actor 연결, 이벤트 배분, 전체 화면 모델 조립	UiApplicationFacade.ts , App.tsx
UI/src/app/runtime, hooks	React 구독과 actor 수명 관리, 데스크톱 창 종료 연결	UiApplicationStore.ts , useUiApplication.ts
UI/src/app/machines	메인/상세 화면 이동과 공통 모달 점유 상태	uiShellMachine.ts
UI/src/features/*/machines	기능별 현재 상태, 이벤트, 가드, Action, 비동기 완료/실패 전이	아래 12개 actor 대응표
UI/src/features/*/components, UI/src/routes	버튼, 차트, 팝업, 거래 내역 등 실제 React 화면	App.tsx 에서 각 화면으로 연결
UI/src/app/presenters	화면의 입력을 intent로 변환하고 표시용 속성과 callback 구성	dashboardPresenter.ts , tradeHistoryPresenter.ts , csvExportPresenter.ts
UI/src/shared/ports, shared/api	외부 명령의 계약, HTTP 요청, WebSocket 이벤트 해석	UiCommandPort.ts , BackendUiAdapter.ts , backendEventMapper.ts
UI/apps/desktop/src-tauri/src	네이티브 폴더 선택, 창과 backend 프로세스	dialog.rs , UI 쪽 연결은 NativeSidecarLifecycle.ts

폴더	책임	읽기 시작할 파일
	의 실행 경계	
backend/src/binance_auto_trader	실제 매매 시작·중지, 체결 저장, 내역 조회와 CSV 파일 쓰기	trading.py, trade_history_controller.py

문서에 나오는 상대 폴더 표기는 프로젝트 루트를 기준으로 합니다. 파일 링크는 실제 절대 경로와 조사 시점의 줄 번호로 연결했습니다.

2. UI의 “STM 객체”는 정확히 무엇인가

교수님, Python의 `TradingSTM`과 UI의 상태 머신은 실행 방식이 다릅니다. UI에서는 아래 세 가지를 구분해야 합니다.

1. `create_chart_machine()` 같은 함수가 만드는 **machine 정의**에는 상태·이벤트·가드·Action 규칙이 들어 있습니다.
2. `createActor(machine)`으로 만든 **actor 객체**가 실제 현재 상태와 context를 보관하고, `.send(event)`를 받아 정의된 규칙을 실행합니다.
3. **UiApplicationFacade 객체**가 actor들을 생성·시작·구독하고, 어떤 actor에게 어떤 이벤트를 보낼지 조정합니다.

`UiApplicationActors`와 `UiApplicationFacade` 생성자에 실제 구성이 있습니다.

Facade의 속성	표 ID	상태 머신 파일과 맡은 책임
actors.shell	ES2-01~03	uiShellMachine.ts: 화면 이동, history 노드, 공통 모달 슬롯
actors.connection	U1-01~03	connectionMachine.ts: 연결·재연결·연결 실패 표시
actors.trading	U2-01~10, U3-01~12	tradingCommandMachine.ts: 자동매매 시작·중지·강제 매도 확인 및 처리 상태
actors.regime	R1-01~02, R2-01~05, R3-01~02	regimeMachine.ts: 추천값, 후보/적용 type, 지표, 선택 확인
actors.chart	DC1~DC6, IP1~IP3	chartMachine.ts: 차트의 6개 Region과 지표 설정 내부 3개 Region
actors.account_summary	DI1-01~02, DI2-01~02	accountSummaryMachine.ts: 투자 정보·자산 정보 2개 Region
actors.split_order	SI-01~02, SO-01~02	splitOrderMachine.ts: 분할 매수·매도 비율과 저장 명령
actors.recent_orders	M4-01~09	recentOrdersMachine.ts: 체결/실시간 지표 탭, 각 표시 데이터
actors.trade_history_summary	D1~D4	tradeHistorySummaryMachine.ts: 수익률·매도 성과·ETH 보유량·수수료 4개 Region
actors.trade_history	TD2-01~13, TD3-01~07	tradeHistoryMachine.ts: 기간·매수/매도 결합 필터와 실제 조회 상태

Facade의 속성	표 ID	상태 머신 파일과 맡은 책임
<code>actors.csv_export</code>	TD4-01~09, CR1~CR3	<code>csvExportMachine.ts</code> : CSV 팝업, 설정 3개 Region, 파일 생성 성공·실패
<code>actors.app_exit</code>	ES3-01~09	<code>appExitMachine.ts</code> : 보유 포지션 확인, 청산 결과 대기, 안전 종료

`actors.trading`은 UI의 자동매매 제어 상태입니다. 실제 매수·매도 전략을 판단하는 Python `TradingSTM` 객체와는 다릅니다. 예를 들어 UI의 `running`은 “backend가 자동매매 시작을 확인한 상태”이지, “매수가 이미 체결된 상태”가 아닙니다.

또한 코드의 `meta.spec_ids: ['U3-05', ...]`는 설계 행과 소스의 대응을 적어 둔 메타데이터입니다. 이 문자열이 전이를 선택하거나 명령을 실행하지 않습니다. 실제 실행 규칙은 같은 파일의 `states`, `on`, `guard`, `actions`, `invoke`, `onDone`, `onError`에 있습니다.

3. 설계의 Region을 실제로 어떻게 나누었는가

교수님, 설계 최상위의 “상단 상태 바 / 메인·상세 화면 / 종료”라는 세 Region은 구현에서 하나의 부모 STM 아래에 같은 모양으로 들어가 있지 않습니다. `UiApplicationFacade`가 보관하는 여러 actor의 조합으로 표현했습니다.

```

UiApplicationFacade - 아래 12개 actor를 보관하고 이벤트 전달
├─ shell                    parallel: navigation + modal_slot
├─ connection              연결 상태 하나
├─ trading                 시작·중지 절차의 상태 하나
├─ regime                 선택 절차의 상태 + 추천값·지표 context
├─ chart                   parallel: 6개 Region
│   ├── interval
│   ├── indicator_settings  opened일 때 parallel
│   │   ├── bollinger_bands
│   │   ├── ema9
│   │   └── volume
│   ├── active_state
│   ├── viewport
│   ├── drawing
│   └── line_selection
├─ account_summary         parallel: trading_logic_status + asset_summary
├─ split_order             공통 ready / saving / failed + 비율 두 개
├─ recent_orders           체결 탭 또는 지표 탭
├─ trade_history_summary   parallel: profit_rate / sell_performance /
│                           eth_holdings / daily_trading_fee
├─ trade_history           조회 상태 하나 + period·side context
├─ csv_export              editing일 때 parallel
│   ├── file_browser
│   ├── period
│   └── file_name
└─ app_exit                종료 절차의 상태 하나
  
```

위 나무에서 Facade 아래 연결은 객체를 보관하는 관계입니다. 12개 actor가 하나의 XState 부모 machine의 `states`에 들어 있다는 뜻은 아닙니다. `parallel`이라고 표시한 부분만 실제 machine 정의에 그 설정이 있습니다.

설계의 Region 구분	현재 코드의 구현 방식	해석
최상위 상단/화면/종료 3개	여러 peer actor와 Facade 조정	책임은 나누었지만 설계 계층과 동일하지 않음
상단 API/중지/시작 3개	connection과 trading 2개 actor	시작·중지는 하나의 lifecycle 상태로 합침
REGIME 추천/선택/지표 3개	regime 하나의 선택 상태와 context 필드들	세 데이터를 유지하지만 parallel 세 Region은 아님
차트 6개	chart의 type: 'parallel'과 6개 자식	설계의 직교 상태 구분을 직접 표현
지표 설정 BB/EMA/거래량 3개	indicator_settings.opened의 parallel	설정창이 열린 동안 세 하위 상태가 함께 활성화
계좌 정보/분할 주문	account_summary와 split_order actor	표시와 설정 명령을 분리
분할 매수/매도 2개	비율 필드 두 개, 공통 저장 상태 하나	완전히 독립적인 두 저장 Region은 아님
상세 계좌 정보 4개	trade_history_summary의 parallel	네 표시 영역을 직접 표현
상세 기간/거래 종류 2개	trade_history.context.period, .side + 공통 조회 상태	두 필터를 보존·결합하지만 두 상태 Region은 아님
CSV 경로/기간/파일명 3개	editing의 parallel	세 입력 절차를 직접 표현

예를 들어 차트는 “30분봉”이면서 동시에 “전체화면”이고, “그리기 대기”이며, “선 선택 없음”일 수 있습니다. 하나의 `current_state = '전체화면'` 때문에 봉 주기나 그리기 상태를 잃지 않도록 나눈 것입니다. 이 부분은 사용자가 Region을 설계한 의도와 잘 맞습니다.

4. 여기서 병렬이라는 말은 스레드가 여러 개라는 뜻인가

교수님, 이 구현의 병렬성을 세 가지로 나누어 설명하겠습니다.

구분	실제 의미
논리적 병렬 상태	여러 Region의 상태가 동시에 활성화되어 보존됨. <code>chart</code> 가 대표적
비동기 작업의 동시 진행	한 actor의 HTTP 응답 대기 중 다른 actor가 차트 입력이나 연결 이벤트를 처리할 수 있음
CPU·스레드 병렬 실행	Region마다 JavaScript 스레드를 만들어 전이 함수를 동시에 실행하는 구조는 없음

`Facade.start()`의 실제 시작 코드는 다음과 같습니다.

```
Object.values(this.actors).forEach((actor) => actor.start());
```

이 `forEach`는 actor들을 시작합니다. 이 반복문이 계속 돌면서 Region을 주기적으로 실행하는 것은 아닙니다. 이후 이벤트를 받은 actor를 XState가 처리합니다. UI Region용 `_TradingEventRuntimeWorker`나 0.25초 순회 루프는 없습니다. 이전 문서에서 설명한 Python backend 작업자는 별개의 실행 경로입니다.

동일 actor에서는 XState가 이벤트를 순서대로 처리하고, 그 이벤트로 선택되는 전이·동기 Action·필요한 즉시 전이를 처리해 다음 안정 상태를 만듭니다. `parallel` 내부에서 같은 이벤트를 처리할 Region이 여러 개이면 그 Region들이 같은 이벤트에 반응

할 수 있습니다. 모든 Region에 반드시 대응 전이가 있어야 하는 것은 아닙니다.

다만 **Promise** 작업이 끝날 때까지 모든 UI 이벤트를 막는 것은 아닙니다. 예를 들어 CSV actor가 **exporting** 상태로 들어가 파일 생성 응답을 기다리면, 시작 이벤트에 대한 동기 처리는 끝난 상태입니다. 나중에 Promise가 완료되면 XState가 별도의 완료 이벤트를 처리하여 **onDone** 전이를 실행합니다. 이 대기 중 다른 actor의 처리도 가능합니다. 허용할 사용자 조작은 각 machine의 전이와 Facade의 모달 규칙이 추가로 제한합니다.

또한 12개 actor 전체를 하나의 원자적인 RTC 단위로 묶은 공통 처리기는 없습니다. Facade가 여러 actor에게 **.send()** 하는 코드는 순서대로 호출됩니다. 서버의 같은 시점 데이터를 화면에 함께 공개해야 할 때에는 **notification_batch_depth**를 사용하는 동기화 경로와 전체 **snapshot 동기화**에서 알림을 묶습니다. 이것은 **화면에 중간 조합을 공개하지 않기 위한 조정**이며, 모든 이벤트에 대한 전역 트랜잭션은 아닙니다.

UiApplicationStore.notify_listeners()의 **requestAnimationFrame**도 화면 구독 알림을 묶기 위한 장치입니다. 매매 명령이나 Region 전이를 매 프레임 재실행하는 타이머가 아닙니다. 별도로 REGIME 강조의 **after: 4000**, 상세 조회의 다음 KST 자정 갱신 같은 타이머는 있지만, 그것 역시 Region별 전용 스레드를 의미하지 않습니다.

5. 이벤트는 누가 만들고 누구에게 전달하는가

교수님, 사용자 입력과 backend 통지는 진입 경로가 다릅니다.

[사용자 입력]

React 버튼/차트/팝업의 event handler 또는 presenter callback

- **UiApplicationStore.dispatch(intent)**
- **UiApplicationFacade.dispatch(intent)**
- 선택된 **actors.xxx.send(machine_event)**
- XState가 현재 상태·event·guard를 보고 전이와 Action 실행

[backend 통지]

BackendUiAdapter의 WebSocket 수신·검증

- **map_backend_event_to_intents()**
- **createLiveUiApplication**의 **on_event** callback
- **intents.forEach(intent => facade.dispatch(intent))**
- 선택된 actor들에게 명시적으로 전달

App.tsx에서 **controller**라고 부르는 값은 **use_ui_application()**이 반환하는 **UiApplicationStore** 객체입니다. Python의 **TradingController**가 아닙니다. **Store**의 **dispatch()**가 Facade로 전달합니다.

```
const accepted = this.application?.facade.dispatch(intent) ?? false;
```

반환된 boolean은 이 UI 전달 경로에서 요청을 받아 처리했는지를 나타냅니다. **HTTP 성공이나 실제 주문 체결 성공을 뜻하지 않습니다**. 또한 actor가 현재 상태에서 그 이벤트를 무시하는 경우까지 모두 엄격하게 판정한 전이 성공 증명도 아닙니다. Facade는 모달 충돌 등에서 **false**를 반환하지만, 전달한 이벤트에 실제 전이가 있었는지는 actor의 snapshot으로 확인해야 합니다.

표의 Action도 종류에 따라 수행 위치가 달라집니다.

표의 Action	실제 수행 구조
버튼 색·텍스트·팝업 표시 변경	XState 상태/ assign 으로 context 변경 → Facade의 ViewModel → React 조건부 렌더링과 CSS

표의 Action	실제 수행 구조
차트 주기·지표·선 관리	chart actor의 상태와 context 변경 → presenter/차트 컴포넌트가 해당 데이터와 선 표시
매매 시작·중지, 설정 저장, CSV 생성	machine의 invoke → fromPromise 로 등록한 작업 → UiCommandPort 메서드 → live에서는 BackendUiAdapter
비동기 성공·실패 후 상태 변경	Promise 완료/거절을 XState가 onDone/onError 로 처리
backend에서 이미 변경된 값 표시	WebSocket 또는 snapshot의 값을 Facade가 actor에 전달 → assign 으로 표시 데이터 갱신

Python TradingSTM처럼 **result.actions**를 반환한 뒤 Controller의 **_execute_action()** callback으로 전부 순회하는 구조를 그대로 사용하지 않습니다. UI의 machine Action은 XState가 실행하고, 외부 작업은 주입받은 **command_port**로 위임합니다. 따라서 **.send()**의 반환값에서 Action 목록을 꺼내는 코드도 없습니다.

현재 guard는 machine 정의의 **guards**와 전이의 **guard**에서 확인할 수 있습니다. **context**는 actor가 유지하는 값이고, **event**는 이번 입력입니다. 예를 들어 시작 확인 시 API 상태는 Facade가 connection actor에서 다시 읽어 이벤트에 넣습니다. "선택 type 존재 여부", "API 연결 여부", "CSV 날짜 순서"처럼 원래 표에 적힌 가드를 검사하기 위해서입니다.

6. 상단 상태 바: U1·U2·U3

교수님, API 상태는 **connectionMachine**, 자동매매 시작과 중지는 **tradingCommandMachine**에서 처리합니다. 화면 표시와 팝업은 **App.tsx** 및 **AppModalHost.tsx**로 이어집니다.

표 ID	구현한 처리
U1-01~03	api_offline, connecting, api_online, reconnecting 과 연결 context로 표시. 원문의 두 상태보다 연결 진행·재연결 상태를 더 세분화
U2-01~04	실행 여부와 포지션 여부로 일반 중지 확인, 강제 매도 확인, 미실행 안내를 구분
U2-05~06	확인하면 stopping 에서 stop_trading() 호출, 취소하면 running 복귀
U2-07~10	force_sell_confirmation → force_selling ; 완료·실패에 따라 종료 대기/중지 확정 또는 확인창 복귀
U3-01~04	시작 버튼 입력에서 미선택 type, 사용할 수 없는 전략, API 연결, 열린 포지션 등을 검사해 적절한 팝업 선택
U3-05~07	시작 확인 때 재검사 후 starting 진입. start_trading() 성공 후 running , 실패하면 오류를 가진 확인창 복귀, 취소하면 stopped
U3-08~09	안내 확인으로 팝업 종료. REGIME 선택 안내 확인은 Facade가 regime actor에 HIGHLIGHT_REQUESTED 도 전달
U3-10~12	시작/중지 버튼의 상태를 공통 trading lifecycle로 유지. 연결이 끊어지면 중지 처리·결과 확인 경로를 진행

U3-08의 5회 점등은 **RegimePanel.module.css**의 **animation: regime-highlight 800ms ease-in-out 5**와 regime actor의 강조 상태로 구현합니다. machine이 전구를 다섯 번 직접 그리는 것은 아닙니다.

표에서는 중지 성공 이벤트가 별도 시작 Region에도 전달되는 것처럼 보입니다. 현재 구현은 시작/중지를 같은 **trading actor**에 합쳤기 때문에 공통 **stopped** 상태가 두 버튼의 표시를 함께 결정합니다. **FORCE_SELL_AND_STOP_SUCCEEDED**라는 표의 이름 그대로 다른 시작 actor에 보낼 필요가 없습니다.

실제 backend의 **stop_trading()** 응답은 **stopping**일 수도 있습니다. 따라서 HTTP가 성공했다고 곧바로 “포지션 청산까지 완료”로 처리하지 않고 **awaiting_stop_completion** 등에서 backend lifecycle 확인을 기다립니다. UI 표의 한 행을 네트워크 요청·대기·완료 확인이라는 여러 상태로 세분화한 것입니다.

7. 메인 화면: REGIME·차트·계좌·최근 체결

7.1 REGIME 패널 — R1·R2·R3

regimeMachine.ts의 context에는 추천 type, 선택 후보, 실제 적용 type, 지표 값이 들어 있습니다. 추천 이벤트는 추천값을, 지표 이벤트는 수치 목록을 갱신합니다. type 클릭은 후보를 저장하고 확인 상태로 들어가며, 확정 시 **apply_regime** Promise를 실행합니다. 성공하면 후보를 적용값으로 확정하고, 취소하면 후보를 버려 이전 적용값을 보여줍니다.

여기에는 세 개의 **parallel** Region이 없습니다. 추천·지표를 최상위 이벤트 처리와 context로 유지하면서, 선택 절차만 상태로 나눴습니다. 따라서 선택 확인창이 열려 있어도 추천값을 갱신할 수 있지만, 그 구조를 “추천·선택·지표 세 Region이 코드에 그대로 있다”고 설명하면 부정확합니다.

특히 **R2-03**은 원문과 동작이 다릅니다. 표는 자동매매 정지 중이면 type 클릭 즉시 적용하도록 되어 있지만, **Facade**의 **REGIME_TYPE_CLICKED**와 machine의 **TYPE_CLICKED**는 정지 여부와 무관하게 확인창을 거칩니다. 이는 **UI_Rule.md**의 **CR-03**에 있는 “클릭 즉시 최종 적용하지 않는다”는 규칙과 일치합니다. 따라서 이 항목은 단순히 누락된 코드라고 단정하기보다 두 설계 문서의 규칙이 충돌하며 현재 코드는 **UI_Rule** 쪽 동작을 따른다고 설명해야 합니다.

7.2 차트 — DC1~DC6·IP1~IP3

chartMachine.ts에 설계의 여섯 Region이 가장 직접적으로 구현되어 있습니다.

Region과 표 ID	실제 상태·Action	화면에서 일어나는 일
interval , DC1-01~13	1m/30m/4h/1d , select_1m_interval 등 주기별 Action	선택한 봉 주기로 표시하고 drawings[interval] 의 선 목록 사용
indicator_settings , DC2-01~03	closed/opened	지표 설정창 표시·닫기
내부 BB, IP1-01~05	choice/on/off , BB 표시값 변경	BB 표시와 on/off 버튼 상태 갱신
내부 EMA, IP2-01~05	choice/on/off , EMA 표시값 변경	EMA9 표시와 버튼 상태 갱신
내부 거래량, IP3-01~05	choice/on/off , 거래량 표시값 변경	거래량 표시와 버튼 상태 갱신
active_state , DC3-01~02	displayed , update_active_state	backend에서 받은 현재 전략 상태 문구 표시
viewport , DC4-01~03	normal/fullscreen 과 is_fullscreen	차트 확대·축소
drawing , DC5-01~07	deactivated/waiting/user_drawing , 선 저장	그리기 모드 진입, 진행 중 선 표시, 완료·취소

Region과 표 ID	실제 상태·Action	화면에서 일어나는 일
<code>line_selection</code> , DC6-01~06	<code>awaiting_selection/highlighted/context_menu</code>	선 강조, 우클릭 메뉴, 삭제·메뉴 닫기

표에 “봉과 지표를 가져온다”라고 적힌 부분 전체가 machine 안에 있는 것은 아닙니다. machine은 선택 주기와 표시 옵션을 관리하고, `useRealtimeChartData.ts`와 `PriceChartPanel.tsx·LightweightChartSurface.tsx`가 실제 데이터 수신·렌더링을 맡습니다. 입력 연결은 `dashboardPresenter.ts`에서 확인할 수 있습니다. 아래 예시와 부록은 선택 상태의 전이 위치를 가리킵니다.

Region 간 가드도 존재합니다. `chart`의 `guards`에서 `is_line_selection_idle`은 선택된 선이 없는지를, `is_not_drawing`은 현재 그리는 중이 아닌지를 검사합니다. 각각 `selected_line_id`, `drawing_mode`라는 같은 chart actor의 context를 읽습니다. DC5-02의 “Region6이 선택 대기인지”, DC6-02·03의 “Region5가 그리는 중이 아닌지”를 이 값으로 표현한 것입니다. Region별 클래스가 서로 메서드를 호출하는 구조는 아닙니다.

그리기 결과는 `context.drawings`에 봉 주기별로 보관합니다. 이것은 actor가 살아 있는 동안의 메모리 보존입니다. 이 저장 Action만으로 앱 재시작 후에도 선이 복원되는 디스크 영속 저장이 구현되었다고 볼 수는 없습니다.

7.3 계좌와 분할 주문 — D11·D12·SI·SO

`accountSummaryMachine.ts`는 `trading_logic_status`와 `asset_summary`를 `parallel`로 둡니다. 표시값 갱신 이벤트에 `assign` Action이 반응하고 React 카드가 바뀝니다. UI가 이 Action에서 거래 수익을 다시 계산하거나 거래소 계좌를 직접 조회하는 것은 아닙니다. live 초기화와 backend 통지가 준비한 표시 데이터를 받습니다.

`splitOrderMachine.ts`는 두 비율을 `scale_in_percentage`, `scale_out_percentage`로 각각 보관합니다. 입력하면 `prepare_scale_in` 또는 `prepare_scale_out`이 요청값을 준비하고, 공통 `saving` 상태에서 `command_port.update_split_order(side, percentage)`를 호출합니다.

따라서 매수·매도 비율을 따로 설정하려는 의도는 구현됐지만, 두 저장 작업의 진행 상태까지 독립 Region으로 나뉘지는 않았습다. 공통 `pending_side`, `pending_percentage`, `error`를 사용합니다. `saving` 중 다른 입력은 `reenter: true`로 새 저장 invoke를 시작할 수 있습니다. 이전 invoke가 비활성화되는 것과 이미 전송된 HTTP 쓰기가 실제로 취소되는 것은 별개이므로, 이를 “두 Region의 완전한 독립 저장 보장”이라고 설명할 수는 없습니다.

7.4 최근 체결·실시간 지표 — M4-01~09

`recentOrdersMachine.ts`의 상태는 `trade_history_displayed`와 `realtime_indicator_displayed`입니다. 두 탭 모두 체결 이벤트를 받을 때 `prepend_trade`로 actor의 `trades` 앞에 거래를 붙입니다. 지표 탭을 보고 있어도 체결 목록은 갱신되므로 돌아왔을 때 목록을 보여줄 수 있습니다. 전략별 실시간 지표는 별도 context에 보존합니다.

여기서 원문의 “체결 내역을 저장한 파일에 저장”은 UI actor의 파일 쓰기로 구현되어 있지 않습니다. 실제 저장은 backend `TradeHistoryController._publish_completed_trade()`와 `trade_history_repository.py`가 맡고, UI는 그 결과를 snapshot과 이벤트로 받습니다. 파일 읽기도 live 초기화에서 받은 내역으로 시작합니다. 탭 전환 때마다 UI가 거래 파일을 다시 여는 구조는 아닙니다.

8. 상세 내역 화면: 요약·결합 필터·CSV

8.1 상세 요약 — D1~D4

`tradeHistorySummaryMachine.ts`는 `profit_rate`, `sell_performance`, `eth_holdings`, `daily_trading_fee`를 병렬 Region으로 둡니다. 각 Region은 표시 상태를 유지하며 자기 데이터만 갱신합니다.

다만 실제 live 이벤트 경로에서는 표의 `SELL_ORDER_EXECUTED` 하나를 모든 요약 Region에 무조건 방송하지 않습니다. `backendEventManager.ts`는 `ORDER_EXECUTED`를 최근 체결용 intent로, `PERFORMANCE_UPDATED`를 성과 갱신용

intent로 변환합니다. 계좌·세션 snapshot도 별도로 들어옵니다. **Facade의 성과 전달 경로** 등이 서버가 확정한 요약값을 적절한 actor에 보냅니다. machine에 표의 이벤트 이름이 존재하는 것과, 실제 서버 이벤트가 같은 이름으로 직접 도착하는 것은 구분해야 합니다.

8.2 기간·매수/매도 결합 필터 — TD2·TD3

교수님, 이 부분은 설계의 두 Region을 **context 필드 두 개와 조회 상태 하나로** 표현했습니다.

```
trade_history.context.period = today / last7days / last30days / all
trade_history.context.side   = all / buy / sell
trade_history의 상태        = idle / loading / ready / empty / failed
```

기간을 바꾸는 Action은 **period**만 바꾸고, 거래 종류를 바꾸는 Action은 **side**만 바꿉니다. **loading의 invoke**는 두 값을 아래처럼 한 요청에 넣습니다.

```
query: {
  period: context.period,
  side: context.side,
},
```

command_port.load_trade_history(query, signal)이 **backend의 get_trades()**와 연결되고, 성공하면 **store_records**로 목록을 넣습니다. 최초 진입은 오늘·전체로 초기화하며 이후 재진입은 현재 query를 유지합니다. 조회 중 나간 경우 invoke의 **AbortSignal**로 HTTP 읽기를 중단합니다. 조회 중 새 체결을 받으면 **refresh_pending**을 기록하고 응답 뒤 다시 조회하여 체결 이전의 목록을 최종 결과로 남기지 않도록 합니다.

현재 필터 선택 전이는 **ready·empty·failed**에 있고 **loading**에는 없습니다. 따라서 “조회 중에도 두 필터 Region이 각각 자유롭게 전이한다”는 원문 그대로의 구조는 아닙니다. 화면의 로딩 처리와 함께 봐야 합니다. 대신 두 필터를 결합하여 하나의 일관된 조회 결과로 관리하려는 의도는 분명하게 구현되어 있습니다.

8.3 CSV 팝업과 내부 Region — TD4·CR1·CR2·CR3

csvExportMachine.ts의 바깥 상태는 **closed → editing → exporting → complete/error**입니다. **editing** 내부만 실제 **parallel 3개 Region**으로 구성했습니다.

표 ID	구현
TD4-01~03	열기에서 reset_draft , 닫기에서 closed 전이. 기본 날짜는 팝업을 여는 시점의 KST 날짜로 준비
TD4-04~05	EXPORT_CSV 에서 is_csv_draft_valid 검사. 통과하면 exporting , 실패하면 validate_draft 로 오류 표시
TD4-06~09	파일 생성 Promise의 onDone/onError 로 완료/실패. 실패 확인 시 기존 draft를 가진 editing , 완료 확인 시 closed
CR1-01~04	file_browser.closed/opened; pick_csv_directory() 의 문자열 결과면 경로 저장, null 이면 기존 경로 유지
CR2-01~13	period.today/weekly/monthly/custom ; 날짜 범위 및 달력 입력 활성 상태 변경

표 ID	구현
CR2-14~21	<code>custom.idle/start_calendar/end_calendar</code> ; 시작·종료일 guard, 잘못된 날짜는 오류를 표시하며 달력 유지
CR3-01~07	<code>file_name.committed/editing</code> ; 입력 draft와 확정값 분리, 유효한 Enter/포커스 해제에 확정, 잘못된 입력은 편집 상태 유지

파일명의 `DEFAULT_FILE_NAME`과 `FILE_NAME_WRITTEN`은 현재 코드에서 모두 `committed`로 합쳤습니다. 기본값인지 수정된 값인지는 context의 내용으로 구분됩니다.

폴더 선택은 Rust의 `choose_csv_export_directory()`까지 연결됩니다. CSV 파일 생성은 `BackendUiAdapter.export_csv()` → `create_csv_export()` → `TradeHistoryController.export_csv()` → `CSVFileGateway.write_csv()` 순서입니다. **CSV actor**는 파일 생성 요청과 진행 상태를 관리하고 실제 **CSV 바이트**는 **backend**가 파일에 씁니다. backend는 날짜와 파일명 등을 다시 검증한 후 실제 파일 생성 결과를 반환합니다.

9. 메인으로 돌아올 때 H*는 무엇을 복원하는가 — ES2

교수님, `uiShellMachine`에는 실제로 다음 history 노드가 있습니다.

```
dashboard_history: {
  type: 'history',
  history: 'deep',
  target: 'dashboard',
  meta: { spec_ids: ['ES2-03', 'H*'] },
},
```

그러나 이 노드는 shell의 `navigation` 안에 있습니다. 차트·REGIME·최근 체결 actor들은 이 history 노드의 하위 상태가 아닙니다. 따라서 *이 H 하나가 모든 feature actor의 상태를 저장·복원하는 것은 아닙니다.**

Facade의 화면 이동 처리는 shell의 route와 상세 조회 actor의 진입/이탈을 변경합니다. 메인에서 상세 화면으로 갔다고 chart actor를 `.stop()`하거나 새로 만들지 않습니다. 그래서 차트 주기·지표 선택·완성된 선·최근 체결 탭 같은 값은 기존 actor의 context에 남습니다. 이 객체 수명 유지가 화면 복귀 시 상태 보존의 중요한 실제 원리입니다.

다만 React 컴포넌트 내부에만 있는 일시적 상태까지 모두 보존된다는 뜻은 아닙니다. 또한 숨겨진 화면의 actor도 backend 이벤트 받을 수 있으므로, 복귀 시점에는 “떠날 때 정지시켜 둔 모든 데이터”가 아니라 **유지된 사용자 설정과 갱신된 서버 데이터를** 보여줄 수 있습니다. 설계의 거대한 `MAIN_SCREEN_WRAPPER` 전체에 대한 deep history와 정확히 같은 구조라고 볼 수 없습니다.

10. 프로그램 종료 Region — ES3

교수님, 실제 OS 창 닫기 요청은 `use_desktop_window_lifecycle()`이 `preventDefault()`로 즉시 종료를 막은 뒤 `APP_EXIT_CLICKED` intent로 바꿉니다. `UiApplicationFacade`는 현재 trading actor의 포지션과 실행 여부를 읽어 `app_exit` actor에 `EXIT_CLICKED` 이벤트를 보냅니다.

```
OS 창 닫기
→ use_desktop_window_lifecycle()
→ UiApplicationStore.dispatch(APP_EXIT_CLICKED)
→ UiApplicationFacade.dispatch()
→ actors.app_exit.send(EXIT_CLICKED + 포지션·실행 정보)
→ 확인창
```

- 필요하면 청산 명령 및 실제 종료 결과 대기
- `shutdown_application()`
- `appExitMachine.ui_final_state`
- `use_desktop_window_lifecycle()`가 실제 창 `destroy()`

`appExitMachine.ts`는 포지션이 있으면 `force_sell_exit_confirmation`, 없으면 `exit_confirmation`으로 갑니다. 청산 요청 응답이 `stopping`이면 `awaiting_liquidation_terminal`에서 backend의 terminal 상태와 포지션 종료를 확인합니다. 이때 실패나 조정 필요 상태면 종료를 진행하지 않고 오류를 보여줍니다. 확정된 종료 준비는 `BackendUiAdapter.shutdown_application()`과 네이티브 프로세스 수명 경계로 이어집니다.

`ui_final_state`의 `type: 'final'`은 `app_exit actor`의 최종 상태입니다. 그것만으로 12개 actor가 하나의 공통 부모 `final`로 전이하는 것은 아닙니다. 네이티브 창 종료는 hook이, 앱 수명 종료 때 actor들의 `.stop()`은 `Facade.stop()`과 Store의 `deactivate` 경로가 말합니다. 브라우저 실행에서는 네이티브 창 port가 없으므로 같은 방식으로 브라우저 자체를 닫지는 않습니다.

11. 실제 코드로 따라가는 예시: 자동매매 시작과 응답

교수님, 이미 적용된 REGIME이 있고 API가 연결되어 있으며, backend가 시작 가능한 상태라고 가정하겠습니다. 다음은 실제 소스의 관련 부분을 발췌한 것입니다. 설명에 필요 없는 분기는 생략했습니다.

11.1 버튼이 만드는 것은 UI intent입니다

`App.tsx`에서 시작 버튼에 넘기는 callback입니다.

```
onStartRequested={() => controller.dispatch({ type: 'START_TRADING_CLICKED'
})}
```

여기서 함수 본문은 사용자가 버튼을 눌렀을 때 실행됩니다. 이 `controller`는 `UiApplicationStore`이고, Store가 `UiApplicationFacade.dispatch()`를 호출합니다. callback을 등록했다고 자동매매 명령이 즉시 실행되는 것은 아닙니다.

11.2 Facade가 다른 actor의 값을 읽어 machine 이벤트를 만듭니다

`START_TRADING_CLICKED` 처리에서 읽는 실제 값과 보내는 실제 이벤트는 다음과 같습니다.

```
const regime = this.actors.regime.getSnapshot().context.applied_regime;
const is_online = this.actors.connection.getSnapshot().matches('api_online');

// 열린 포지션, 시작 가능 여부, 모달 충돌 등의 검사 부분 생략
this.actors.trading.send({
  type: 'START_BUTTON_CLICKED',
  regime,
  is_online,
});
```

이 `this`는 `UiApplicationFacade`입니다. `.send()`의 대상은 `actors.trading`이라는 XState actor입니다. actor가 `stopped`일 때 등록된 guard들을 순서대로 검사하고, 조건이 맞으면 `start_confirmation`으로 들어갑니다. 이것이 표의 U3-02에 해당하는 동작입니다. 아직 `start_trading()`은 호출하지 않았습니다.

11.3 확인 버튼이 별도의 이벤트를 보냅니다

`AppModalHost.tsx`에는 다음 callback이 있습니다.

```
onConfirm={() => controller.dispatch({ type: 'START_TRADING_CONFIRMED' })}
```

Facade는 이 intent를 아래 이벤트로 바꿉니다.

```
this.actors.trading.send({
  type: 'START_CONFIRMED',
  is_online: this.actors.connection.getSnapshot().matches('api_online'),
});
```

API 연결을 다시 읽는 이유는 확인창을 연 뒤 연결이 끊겼을 수도 있기 때문입니다. machine은 `start_confirmation`의 가드를 다시 검사하고 시작 가능하면 `starting`으로 이동합니다.

11.4 starting에 들어가면 XState가 비동기 명령을 시작합니다

`tradingCommandMachine`의 작업 등록은 다음과 같습니다.

```
start_trading: fromPromise<TradingCommandReceipt, RegimeType>(async ({ input }) => {
  return command_port.start_trading(input);
}),
```

여기의 `command_port`는 `create_trading_command_machine(command_port, ...)`을 호출할 때 전달된 객체입니다. live 구성에서는 `createLiveUiApplication`이 만든 `BackendUiAdapter` 객체입니다. 이 함수는 그 객체를 클로저로 참조합니다. 그러므로 XState가 등록된 작업을 실행하면 실제로 `BackendUiAdapter.start_trading(input)`이 호출됩니다.

`starting` 상태의 `invoke`가 어떤 작업을 호출할지 연결합니다.

```
invoke: {
  id: 'start_command',
  src: 'start_trading',
  input: ({ context }) => context.selected_regime as RegimeType,
  onDone: {
    target: 'running',
    actions: 'mark_trading_started',
  },
  onError: {
    target: 'start_confirmation',
    actions: 'remember_failure',
  },
},
```

`src: 'start_trading'`은 앞서 `actors`에 등록한 작업 이름을 가리킵니다. `input` callback은 현재 선택된 REGIME 값을 그 작업에 넘깁니다. `onDone`과 `onError`는 성공·실패 시 수행할 전이 설정이며, 사용자가 직접 호출하는 함수 이름이 아닙니다.

다.

11.5 실제 backend 호출과 결과 확인

`BackendUiAdapter.start_trading()`은 `POST /v1/trading/start` 요청을 보내며 `expected_version`을 함께 보냅니다. 응답에 유효한 `running` 상태와 session ID가 있는지, 응답 version이 적절한지 검사합니다. 실패하면 Promise가 reject됩니다.

backend에서는 `transport/routes/trading.py`의 `start_trading()`이 `TradingController.start_trading()`을 호출합니다. 여기부터 Python TradingSTM 초기화와 Action 처리가 이루어집니다. UI는 Python 객체를 직접 참조하지 않고 HTTP 계약을 통해 연결됩니다.

따라서 표의 U3-05는 실제로 아래처럼 나뉩니다.

```
START_CONFIRMED
→ UI actor: starting
→ BackendUiAdapter: HTTP 요청
→ backend TradingController: 실제 시작 처리
→ Adapter: 응답 검증
→ XState: Promise 완료에 따른 onDone
→ UI actor: running, mark_trading_started
```

실패하면 `remember_failure`로 오류를 저장하고 확인 상태로 돌아갑니다. 클릭했다는 사실만으로 계속 실행 중이라고 표시하지 않도록 만든 구조입니다. 서버의 lifecycle WebSocket 통지도 별도로 들어오므로, 응답 Promise와 서버 snapshot을 모두 고려하는 동기화 코드가 추가되어 있습니다.

11.6 “응답대로 화면이 행동한다”는 마지막 연결

Facade 생성자의 구독은 실제로 다음과 같습니다.

```
this.actor_subscriptions.push(actor.subscribe(() =>
this.handle_actor_update()));
```

여기서 `actor`는 순회 중인 XState actor이고, 화살표 함수가 참조하는 `this`는 `UiApplicationFacade`입니다. actor의 snapshot이 바뀌면 XState가 구독 callback을 호출하고, Facade는 `handle_actor_update()`에서 모달 상태와 ViewModel 알림을 조정합니다.

```
actor snapshot 변경
→ Facade.handle_actor_update()
→ 모달 슬롯 조정, snapshot/화면 모델 공개
→ UiApplicationStore 구독 callback
→ useSyncExternalStore
→ App/AppModalHost/각 React 컴포넌트 다시 렌더링
```

이것이 시작 버튼이 “자동매매 실행 중”으로 바뀌고 확인창이 사라지는 실제 구조입니다. machine이 React 버튼의 텍스트를 직접 변경하는 코드는 없습니다.

12. 실제 코드로 보는 Region 독립성과 여러 Action

12.1 EMA만 꺼도 다른 지표는 그대로 유지됩니다

아래는 `chartMachine.test.ts`에 있는 실제 검증입니다.

```
const actor = createActor(create_chart_machine());

actor.start();
actor.send({ type: 'INDICATOR_SETTINGS_BUTTON_CLICKED' });
actor.send({ type: 'EMA_DISPLAY_OFF_CLICKED' });

expect(actor.getSnapshot().context.indicators).toEqual({
  bollinger_bands: true,
  ema9: false,
  volume: true,
});
actor.stop();
```

설정창을 열면 내부 BB·EMA·거래량 Region이 함께 활성화됩니다. EMA 끄기 이벤트에는 EMA Region이 반응하고, 해당 Action은 다른 필드를 보존하면서 `ema9`만 `false`로 만듭니다. BB와 거래량은 유지됩니다. 이는 독립적인 상태를 동시에 유지한다는 의미의 병렬성이지만, EMA 작업 스레드가 실행되었다는 뜻이 아닙니다.

실제 사용자 입력은 `Facade.dispatch_chart_indicator()`가 `CHART_INDICATOR_CHANGED`의 종류와 boolean을 `EMA_DISPLAY_OFF_CLICKED` 같은 구체 이벤트로 바꾸어 보냅니다.

차트 선 선택과 그리기는 완전히 무관하지 않습니다. 앞에서 설명한 guard로 서로의 상태를 제한합니다. 즉 “Region이 독립적”이라는 말은 상태를 따로 보관한다는 뜻이며, 다른 Region의 상태를 조건으로 사용하지 않는다는 뜻은 아닙니다.

12.2 CSV 내보내기에서는 설정 Region 전체를 빠져나와 처리 상태로 갑니다

`csvExportMachine`의 `editing` 상태에는 실제로 다음 코드가 있습니다.

```
EXPORT_CSV: [
  { guard: 'is_csv_draft_valid', target: 'exporting' },
  { actions: 'validate_draft' },
],
```

예를 들어 경로 미선택 상태에서 이 이벤트를 보내면 `validate_csv_context()` 검사에 실패합니다. 이때 두 번째 분기의 Action이 오류를 context에 저장하고 `editing`에 남습니다. 파일 생성 명령은 아직 호출되지 않습니다.

경로·파일명·날짜가 유효하면 `exporting`으로 갑니다. 이 상태는 `editing`의 형제이므로 경로·기간·파일명 세 Region의 활성화 상태를 함께 빠져나옵니다. `exporting`의 `invoke`가 context의 옵션을 한 묶음으로 `export_csv` 작업에 전달합니다. Promise가 성공하면 `store_receipt` 후 `complete`, 실패하면 `remember_export_failure` 후 `error`입니다.

`CSV_EXPORT_SUCCEEDED·CSV_EXPORT_FAILED`라는 표의 완료 이벤트는 현재 구현에서 해당 Promise의 `onDone·onError`로 대응됩니다. 표의 이벤트 문자열을 그대로 `.send()` 하는 코드가 없다고 곧바로 미구현으로 볼 수 없는 이유입니다.

12.3 한 이벤트에 Action이 여러 개 있어도 따로 Controller 반복문을 만들지 않습니다

상세 내역 조회 성공 분기에는 다음 설정이 있습니다.

```
{
  guard: 'has_records',
  target: 'ready',
  actions: ['store_records', 'publish_summary'],
},
```

`tradeHistoryMachine`에서 XState가 이 Action 배열을 순서대로 처리합니다. `store_records`는 actor의 목록을 갱신하고, `publish_summary`는 허용된 결과에 한해 주입받은 `on_details_loaded` callback을 호출합니다. Facade가 이 callback에서 더 최신 summary revision이 있는지 확인하고 요약 actor에 전달합니다.

여기의 Action 배열 실행 주체는 XState입니다. 예전 Python 예시의

`RunToCompletionEventProcessor._action_executor`를 사용하는 것이 아닙니다. 또한 이 배열이 원문 “팝업 제거, 표시 변경” 같은 모든 시각 효과를 직접 순서대로 그린다는 뜻도 아닙니다. 화면은 최종 snapshot에 대응하여 렌더링됩니다.

13. 설계와 구현이 같은지에 대한 평가

교수님, 현재 구현은 설계의 기능을 폭넓게 반영했고, 특히 차트·지표 설정·상세 요약·CSV 입력의 직교 상태는 코드에서도 분명하게 확인됩니다. 그러나 “182개 ID가 모두 있으니 원문 STM과 완전히 동일하다”는 결론은 내릴 수 없습니다. 다음을 구분해야 합니다.

항목	판단과 근거
전체 ID 추적	182개 모두 machine의 <code>meta.spec_ids</code> 에서 찾을 수 있음. 실행 동작의 완전한 동치 증명은 아님
최상위 Region 계층	원문의 단일 계층 STM을 12개 actor와 Facade로 재구성
차트 6개 및 내부 지표 3개	실제 <code>parallel</code> 로 구현. 사용자 설정을 서로 보존하는 의도가 직접 드러남
REGIME 선택 R2-03	정지 중 즉시 적용이라는 표와 다름. 코드·UI_Rule CR-03은 확인 후 적용
시작·중지 Region	하나의 trading actor로 합쳐 상반된 버튼 상태를 공통 lifecycle로 관리
분할 설정·거래 필터 Region	독립 필드를 보존하지만 저장/조회 처리 상태는 공유
완료·실패 이벤트	일부 표의 명시적 이벤트 문자열을 Promise <code>onDone/onError</code> 와 backend snapshot으로 구현
H* 복귀	history 노드는 navigation 범위. 기능 설정 보존은 주로 actor 수명 유지로 구현
체결 파일 저장	UI actor 대신 backend Controller·repository가 수행
초기 상태	원문의 초기 표시뿐 아니라 live snapshot으로 기존 계좌·세션을 복원. 항상 빈 값으로 시작하지 않음
종료 final	종료 actor의 final, 창 destroy, 전체 actor stop의 책임을 분리
비동기 보강	시작/중지 pending, version 검사, 재연결, 조회 취소, 오류·조정 대기 등을 추가

형식적으로 설계와 구현을 일치시키려면 우선 R2-03과 UI_Rule의 충돌을 정리하고, 최상위 actor 구성·공통 조회/저장 상태·비동기 완료 이벤트를 설계에 명시할 필요가 있습니다. 현재 문서는 그 차이를 설명하며, 이번 작업에서는 코드의 동작을 변경하지 않았습니다.

14. 확인한 범위와 검증 결과

소스의 machine 정의, Facade 이벤트 전달, Store 구독, live adapter, 주요 React 연결, backend 시작·저장·CSV 경로를 대조했습니다. 기존 테스트 중 기능별 machine, Facade, shell, Store, App, live bootstrap을 대상으로 **15개 테스트 파일의 113개 테스트가 통과했습니다.**

대표 근거는 [chartMachine.test.ts](#)의 지표 독립성과 선 보존, [UiApplicationFacade.test.ts](#)의 actor 연결, [tradingCommandMachine.test.ts](#)의 명령 처리, [tradeHistoryMachine.test.ts](#)의 조회, [csvExportMachine.test.ts](#)의 검증·비동기 흐름, [appExitMachine.test.ts](#)의 종료 절차입니다.

이는 관련 기존 테스트의 통과 결과입니다. 이번 작업에서 182개 행마다 새로운 행동 테스트를 작성한 것은 아니며, 실제 거래소 주문이나 네이티브 창 종료·CSV 내보내기를 실행하여 검증한 것도 아닙니다. 색인에 ID가 있다는 사실, 구현을 읽어 확인한 내용, 테스트가 검증한 범위를 구분했습니다.

15. 전체 182개 ID별 소스 색인

아래 표는 원문의 순서를 따릅니다. **구현 표식 링크는 해당 ID가 등록된 meta.spec_ids 위치이며, 실행을 결정하는 문장 자체와 다를 수 있습니다.** 각 묶음의 **실행 정의** 링크와 본문의 설명을 함께 읽으면 실제 **on/guard/actions/invoke**로 이어집니다. 같은 상태에 여러 행을 묶어 표기했거나, 여러 상태가 하나의 설계 행을 나누어 구현한 경우가 있습니다.

None은 원문의 초기 진입 또는 choice 상태의 이벤트 없는 가드 분기입니다. 모든 초기 표시가 개별 Action 함수로 구현된 것은 아니며 **initial/context**, live snapshot 주입, React 최초 렌더링으로 나누어 구현됩니다. 차이가 있는 행에는 아래에서도 표시했습니다.

U1 — API 상태

실행 정의: [connectionMachine.ts](#)

ID	원문의 EVENT	구현 표식	실제 처리·차이
U1-01	None	80	기본 offline context와 상단 표시
U1-02	API_CONNECTED	99 · 115	api_online 전이·mark_online
U1-03	API_DISCONNECTED	80 · 126	현재 코드는 먼저 reconnecting; 원문의 즉시 offline보다 세분화

U2 — 매매 중지

실행 정의: [tradingCommandMachine.ts](#)

ID	원문의 EVENT	구현 표식	실제 처리·차이
U2-01	None	694	공통 trading 상태와 상단 중지 버튼 렌더링
U2-02	STOP_BUTTON_CLICKED	860	running에서 포지션 없음 → stop_confirmation
U2-03	STOP_BUTTON_CLICKED	912	running에서 포지션 있음 → force_sell_confirmation
U2-04	STOP_BUTTON_CLICKED	694	stopped에서 mark_not_running; 복구 포지션이면 별도 청산 확인
U2-05	STOP_CONFIRMED	860 · 881	stopping invoke; terminal이면 stopped, 아니면 완료 대기

ID	원문의 EVENT	구현 표식	실제 처리·차이
U2-06	STOP_CANCELED	860	확인 취소 → running
U2-07	FORCE_SELL_AND_STOP_CONFIRMED	912 · 933	force_selling invoke로 backend 중지 요청
U2-08	FORCE_SELL_AND_STOP_SUCCEEDED	933	Promise 완료와 lifecycle로 terminal 확인 후 stopped
U2-09	FORCE_SELL_AND_STOP_FAILED	912 · 933	onError/결과 판정에 따라 오류 보존·확인창 복귀
U2-10	FORCE_SELL_AND_STOP_CANCELED	912	강제 매도 확인 취소 → running

U3 — 자동매매 시작

실행 정의: [tradingCommandMachine.ts](#)

ID	원문의 EVENT	구현 표식	실제 처리·차이
U3-01	None	694	공통 trading 상태와 시작 버튼 표시
U3-02	START_BUTTON_CLICKED	775	guard 검사 후 start_confirmation
U3-03	START_BUTTON_CLICKED	736	REGIME 없음 → select_regime_notice
U3-04	START_BUTTON_CLICKED	751	API 연결 안 됨 → api_connection_required
U3-05	START_CONFIRMED	775 · 812 · 837	starting invoke 성공 후 running; 실패 분기도 존재
U3-06	START_CONFIRMED	751 · 775	확인 시 API 재검사 → 연결 필요 안내
U3-07	START_CANCELED	775	시작 확인 취소 → stopped
U3-08	SELECT_REGIME_NOTICE_CONFIRMED	736	안내 달기 + Facade가 regime 강조 이벤트 전달
U3-09	API_CONNECTION_NOTICE_CONFIRMED	751	API 안내 달기; 현재 backend lifecycle에 맞는 상태 복귀
U3-10	FORCE_SELL_AND_STOP_SUCCEEDED	837	별도 성공 문자열 방송 대신 공통 trading lifecycle로 버튼 갱신
U3-11	STOP_CONFIRMED	837	중지 확인 즉시 완료가 아니라 backend 결과 후 버튼 갱신

ID	원문의 EVENT	구현 표식	실제 처리·차이
U3-12	API_DISCONNECTED	751 · 1028	disconnect_stopping과 backend 결과 확인 경로

ES2 — 화면 이동

실행 정의: [uiShellMachine.ts](#)

ID	원문의 EVENT	구현 표식	실제 처리·차이
ES2-01	None	83	navigation의 dashboard와 최초 화면 모델
ES2-02	SHOW_ALL_TRADING_DETAILS	83 · 92	shell → trade_history; Facade가 상세 조회 진입도 전달
ES2-03	BACK_TO_MAIN_SCREEN	92 · 104	dashboard_history 경유 복귀; 기능 설정은 기존 actor 수 명으로 유지

R1 — REGIME 추천

실행 정의: [regimeMachine.ts](#)

ID	원문의 EVENT	구현 표식	실제 처리·차이
R1-01	None	207	초기 snapshot/options의 추천값 표시
R1-02	TYPE_RECOMMENDED	207	TYPE_RECOMMENDED가 추천 context 갱신

R2 — REGIME 선택

실행 정의: [regimeMachine.ts](#)

ID	원문의 EVENT	구현 표식	실제 처리·차이
R2-01	None	207	초기 선택 표시; live는 기존 선택값 복원 가능
R2-02	TYPE_CLICKED	243	후보 저장 → type_change_confirmation
R2-03	TYPE_CLICKED	207	원문과 다름: 정지 중에도 확인창을 거침(UI_Rule CR-03)
R2-04	CONFIRM_TYPE_CHANGE	243 · 257	applying invoke 성공 후 후보 확정
R2-05	CANCEL_TYPE_CHANGE	243	후보 취소, 기존 applied_regime 유지

R3 — REGIME 지표

실행 정의: [regimeMachine.ts](#)

ID	원문의 EVENT	구현 표식	실제 처리·차이
R3-01	None	207	초기 snapshot/options의 지표 표시
R3-02	REGIME_INDICATOR_UPDATED	207	REGIME_INDICATOR_UPDATED로 지표 context 갱신

DC1 — 차트 주기

실행 정의: [chartMachine.ts](#)

ID	원문의 EVENT	구현 표식	실제 처리·차이
DC1-01	None	236	기본 30m 또는 주입 interval 상태, 해당 주기 선 표시
DC1-02	1_M_BUTTON_CLICKED	233	interval을 1m으로 변경; 해당 주기의 데이터·선 표시
DC1-03	4_H_BUTTON_CLICKED	239	interval을 4h으로 변경; 해당 주기의 데이터·선 표시
DC1-04	1_DAY_BUTTON_CLICKED	242	interval을 1d으로 변경; 해당 주기의 데이터·선 표시
DC1-05	30_M_BUTTON_CLICKED	236	interval을 30m으로 변경; 해당 주기의 데이터·선 표시
DC1-06	4_H_BUTTON_CLICKED	239	interval을 4h으로 변경; 해당 주기의 데이터·선 표시
DC1-07	1_DAY_BUTTON_CLICKED	242	interval을 1d으로 변경; 해당 주기의 데이터·선 표시
DC1-08	1_M_BUTTON_CLICKED	233	interval을 1m으로 변경; 해당 주기의 데이터·선 표시
DC1-09	30_M_BUTTON_CLICKED	236	interval을 30m으로 변경; 해당 주기의 데이터·선 표시
DC1-10	1_DAY_BUTTON_CLICKED	242	interval을 1d으로 변경; 해당 주기의 데이터·선 표시
DC1-11	1_M_BUTTON_CLICKED	233	interval을 1m으로 변경; 해당 주기의 데이터·선 표시
DC1-12	30_M_BUTTON_CLICKED	236	interval을 30m으로 변경; 해당 주기의 데이터·선 표시
DC1-13	4_H_BUTTON_CLICKED	239	interval을 4h으로 변경; 해당 주기의 데이터·선 표시

DC2 — 지표 설정창

실행 정의: [chartMachine.ts](#)

ID	원문의 EVENT	구현 표식	실제 처리·차이
DC2-01	None	250	indicator_settings.closed
DC2-02	INDICATOR_SETTINGS_BUTTON_CLICKED	259	opened 진입, 내부 지표 세 Region 활성화
DC2-03	INDICATOR_POPUP_OUTSIDE_CLICKED	250	closed 전이; 표시 옵션 context는 유지

IP1 — BB 표시

실행 정의: [chartMachine.ts](#)

ID	원문의 EVENT	구현 표식	실제 처리·차이
IP1-01	None	273	choice가 기존 BB 표시값을 참조
IP1-02	None	283	always의 fallback → off; 기존 BB false 유지
IP1-03	None	292	always의 표시 가드 통과 → on
IP1-04	BB_DISPLAY_ON_CLICKED	283	on 전이와 show Action; 다른 지표 필드는 보존
IP1-05	BB_DISPLAY_OFF_CLICKED	292	off 전이와 hide Action; 다른 지표 필드는 보존

IP2 — EMA 표시

실행 정의: [chartMachine.ts](#)

ID	원문의 EVENT	구현 표식	실제 처리·차이
IP2-01	None	306	choice가 기존 EMA9 표시값을 참조
IP2-02	None	313	always의 fallback → off; 기존 EMA9 false 유지
IP2-03	None	322	always의 표시 가드 통과 → on
IP2-04	EMA_DISPLAY_ON_CLICKED	313	on 전이와 show Action; 다른 지표 필드는 보존
IP2-05	EMA_DISPLAY_OFF_CLICKED	322	off 전이와 hide Action; 다른 지표 필드는 보존

IP3 — 거래량 표시

실행 정의: [chartMachine.ts](#)

ID	원문의 EVENT	구현 표식	실제 처리·차이
IP3-01	None	336	choice가 기존 거래량 표시값을 참조
IP3-02	None	343	always의 fallback → off; 기존 거래량 false 유지
IP3-03	None	352	always의 표시 가드 통과 → on
IP3-04	VOLUME_DISPLAY_ON_CLICKED	343	on 전이와 show Action; 다른 지표 필드는 보존
IP3-05	VOLUME_DISPLAY_OFF_CLICKED	352	off 전이와 hide Action; 다른 지표 필드는 보존

DC3 — 실행 전략 상태

실행 정의: [chartMachine.ts](#)

ID	원문의 EVENT	구현 표식	실제 처리·차이
DC3-01	None	370	초기 active_trading_state 표시
DC3-02	TRADING_LOGIC_STATE_CHANGED	370	update_active_state가 전달된 전략 문구 갱신

DC4 — 전체화면

실행 정의: [chartMachine.ts](#)

ID	원문의 EVENT	구현 표식	실제 처리·차이
DC4-01	None	383	viewport.normal과 is_fullscreen=false
DC4-02	FULL_SIZE_SELECTED	383 · 392	fullscreen 진입 → 확대 표시
DC4-03	NORMAL_SIZE_SELECTED	392	normal 진입 → 축소 표시

DC5 — 선 그리기

실행 정의: [chartMachine.ts](#)

ID	원문의 EVENT	구현 표식	실제 처리·차이
DC5-01	None	406	drawing.deactivated

ID	원문의 EVENT	구현 표식	실제 처리·차이
DC5-02	DRAWING_TOOL_CLICKED	406 · 416	선 선택이 없다는 guard 통과 → waiting
DC5-03	USER_START_DRAWING	416 · 428	user_drawing → 진행 중 그리기 표시
DC5-04	DRAWING_TOOL_CLICKED	416	deactivated → 그리기 비활성
DC5-05	USER_FINISH_DRAWING	428	waiting 복귀 + 완성된 선을 현재 주기의 drawings에 추가
DC5-06	DRAWING_CANCELED	428	waiting 복귀; 진행 중 선 취소
DC5-07	DRAWING_TOOL_CLICKED	428	deactivated 복귀; 진행 중 선 취소

DC6 — 선 선택·삭제

실행 정의: [chartMachine.ts](#)

ID	원문의 EVENT	구현 표식	실제 처리·차이
DC6-01	None	449	awaiting_selection 진입·선택 해제
DC6-02	CURSOR_HOVER_ENTER	449 · 460	그리는 종이 아니면 highlighted + 선 선택
DC6-03	HIGHLIGHTED_LINE_RIGHT_CLICKED	460 · 473	그리는 종이 아니면 context_menu + 메뉴 좌표 저장
DC6-04	CURSOR_HOVER_EXIT	460	awaiting_selection → 선택 해제
DC6-05	DELETE_LINE	473	선 삭제 Action 후 awaiting_selection
DC6-06	CONTEXT_MENU_OUTSIDE_CLICKED	473	awaiting_selection → 메뉴와 선택 종료

DI1 — 투자 상태 정보

실행 정의: [accountSummaryMachine.ts](#)

ID	원문의 EVENT	구현 표식	실제 처리·차이
DI1-01	None	106	초기 strategy context 표시
DI1-02	TRADING_STATUS_UPDATED	106	update_strategy_summary로 투자 정보 갱신

DI2 — 자산 정보

실행 정의: [accountSummaryMachine.ts](#)

ID	원문의 EVENT	구현 표식	실제 처리·차이
DI2-01	None	119	초기 asset context 표시
DI2-02	ASSET_SUMMARY_UPDATED	119	update_asset_summary로 자산 정보 갱신

SI — 분할 매수

실행 정의: [splitOrderMachine.ts](#)

ID	원문의 EVENT	구현 표식	실제 처리·차이
SI-01	None	154	초기 매수 비율 context 표시
SI-02	SCALE_IN_LEVEL_CHANGED	154 · 169 · 203	prepare_scale_in → 공통 saving invoke → 응답 처리

SO — 분할 매도

실행 정의: [splitOrderMachine.ts](#)

ID	원문의 EVENT	구현 표식	실제 처리·차이
SO-01	None	154	초기 매도 비율 context 표시
SO-02	SCALE_OUT_LEVEL_CHANGED	154 · 169 · 203	prepare_scale_out → 공통 saving invoke → 응답 처리

M4 — 최근 체결·지표 탭

실행 정의: [recentOrdersMachine.ts](#)

ID	원문의 EVENT	구현 표식	실제 처리·차이
M4-01	None	114	초기 snapshot/options 내역 표시; UI의 직접 파일 읽기 아님
M4-02	BUY_ORDER_EXECUTED	114	prepend_trade; 실제 파일 저장은 backend
M4-03	SELL_ORDER_EXECUTED	114	prepend_trade; 실제 파일 저장은 backend
M4-04	REALTIME_INDICATOR_CLICKED	114 · 130	realtime_indicator_displayed로 탭 전환
M4-05	REALTIME_INDICATOR_UPDATED	130	update_realtime_indicators 또는 전략 snapshot 동기화
M4-06	TRADING_STATUS_UPDATED	130	전략별 지표 목록 갱신; live는 세션/지표 snapshot 경로도 사용
M4-07	BUY_ORDER_EXECUTED	130	지표 탭 유지 + trades 갱신; 파일 저장은 backend
M4-08	SELL_ORDER_EXECUTED	130	지표 탭 유지 + trades 갱신; 파일 저장은 backend
M4-09	TRADING_HISTORY_CLICKED	130	trade_history_displayed로 전환; actor에 유지된 목록 표시

D1 — 수익률

실행 정의: [tradeHistorySummaryMachine.ts](#)

ID	원문의 EVENT	구현 표식	실제 처리·차이
D1-01	None	161	초기 summary의 수익률 표시
D1-02	PROFIT_RATE_UPDATED	161	수익률 assign; live 성과 snapshot 경로도 사용

D2 — 매도 성과

실행 정의: [tradeHistorySummaryMachine.ts](#)

ID	원문의 EVENT	구현 표식	실제 처리·차이
D2-01	None	172	초기 summary의 매도 성과 표시
D2-02	SELL_ORDER_EXECUTED	172	매도 성과 assign; live PERFORMANCE_UPDATED와 연결

D3 — ETH 보유량

실행 정의: [tradeHistorySummaryMachine.ts](#)

ID	원문의 EVENT	구현 표식	실제 처리·차이
D3-01	None	183	초기 summary의 ETH 수량 표시
D3-02	BUY_ORDER_EXECUTED	183	보유량 assign; live 계좌/snapshot의 확정값 사용
D3-03	SELL_ORDER_EXECUTED	183	보유량 assign; live 계좌/snapshot의 확정값 사용

D4 — 당일 수수료

실행 정의: [tradeHistorySummaryMachine.ts](#)

ID	원문의 EVENT	구현 표식	실제 처리·차이
D4-01	None	195	초기 summary의 당일 수수료 표시
D4-02	DAILY_TRADING_FEE_CHANGED	195	수수료 assign; live 성과 snapshot 경로도 사용

TD2 — 거래 내역 기간

실행 정의: [tradeHistoryMachine.ts](#)

ID	원문의 EVENT	구현 표식	실제 처리·차이
TD2-01	None	329 382	최초 진입에서 today + all로 query 초기화
TD2-02	SELECT_DISPLAY_WEEKLY_HISTORY	382	period=last7days, side 유지 → loading의 결합 query 조회
TD2-03	SELECT_DISPLAY_MONTHLY_HISTORY	382	period=last30days, side 유지 → loading의 결합 query 조회

ID	원문의 EVENT	구현 표식	실제 처리·차이
TD2-04	SELECT_DISPLAY_ALL_HISTORY	382	period=all, side 유지 → loading의 결합 query 조회
TD2-05	SELECT_DISPLAY_TODAY_HISTORY	382	period=today, side 유지 → loading의 결합 query 조회
TD2-06	SELECT_DISPLAY_MONTHLY_HISTORY	382	period=last30days, side 유지 → loading의 결합 query 조회
TD2-07	SELECT_DISPLAY_ALL_HISTORY	382	period=all, side 유지 → loading의 결합 query 조회
TD2-08	SELECT_DISPLAY_TODAY_HISTORY	382	period=today, side 유지 → loading의 결합 query 조회
TD2-09	SELECT_DISPLAY_WEEKLY_HISTORY	382	period=last7days, side 유지 → loading의 결합 query 조회
TD2-10	SELECT_DISPLAY_ALL_HISTORY	382	period=all, side 유지 → loading의 결합 query 조회
TD2-11	SELECT_DISPLAY_TODAY_HISTORY	382	period=today, side 유지 → loading의 결합 query 조회
TD2-12	SELECT_DISPLAY_WEEKLY_HISTORY	382	period=last7days, side 유지 → loading의 결합 query 조회
TD2-13	SELECT_DISPLAY_MONTHLY_HISTORY	382	period=last30days, side 유지 → loading의 결합 query 조회

TD3 — 매수·매도 필터

실행 정의: [tradeHistoryMachine.ts](#)

ID	원문의 EVENT	구현 표식	실제 처리·차이
TD3-01	None	329 382	최초 진입에서 side=all + today query
TD3-02	BUY_TRADE_HISTORY_SELECTED	382	side=buy, period 유지 → loading의 결합 query 조회
TD3-03	SELL_TRADE_HISTORY_SELECTED	382	side=sell, period 유지 → loading의 결합 query 조회
TD3-04	ALL_TRADE_HISTORY_SELECTED	382	side=all, period 유지 → loading의 결합 query 조회
TD3-05	SELL_TRADE_HISTORY_SELECTED	382	side=sell, period 유지 → loading의 결합 query 조회
TD3-06	ALL_TRADE_HISTORY_SELECTED	382	side=all, period 유지 → loading의 결합 query 조회

ID	원문의 EVENT	구현 표식	실제 처리·차이
TD3-07	BUY_TRADE_HISTORY_SELECTED	382	side=buy, period 유지 → loading의 결합 query 조회

TD4 — CSV 실행 절차

실행 정의: [csvExportMachine.ts](#)

ID	원문의 EVENT	구현 표식	실제 처리·차이
TD4-01	None	347	closed: CSV 설정창 대기
TD4-02	CSV_EXPORT_CLICKED	347 · 358	editing + reset_draft
TD4-03	CLOSE_CSV_EXPORT_POPUP	358	closed로 설정창 닫기
TD4-04	EXPORT_CSV	358 · 543	유효한 draft → exporting invoke
TD4-05	EXPORT_CSV	358	validate_draft로 필드 오류 표시; editing 유지
TD4-06	CSV_EXPORT_SUCCEEDED	543 · 568	표의 성공 이벤트는 invoke.onDone → complete
TD4-07	CSV_EXPORT_FAILED	543 · 576	표의 실패 이벤트는 invoke.onError → error
TD4-08	CSV_EXPORT_ERROR_CONFIRMED	576	기존 draft를 유지하고 editing 복귀
TD4-09	ACCEPT_CLOSE_ALL_POPUP	568	완료 확인 → closed

CR1 — 저장 경로

실행 정의: [csvExportMachine.ts](#)

ID	원문의 EVENT	구현 표식	실제 처리·차이
CR1-01	None	373	file_browser.closed
CR1-02	SAVE_LOCATION_SELECT_CLICKED	382	opened 진입 → pick_directory invoke
CR1-03	SAVE_LOCATION_CONFIRMED	382	Promise의 유효 문자열 결과 → store_directory.closed
CR1-04	SAVE_LOCATION_CANCELED	382	Promise의 null 결과 → 경로 유지·closed

CR2 — CSV 기간·달력

실행 정의: [csvExportMachine.ts](#)

ID	원문의 EVENT	구현 표식	실제 처리·차이
CR2-01	None	427	reset_draft에서 오늘 기준 범위 설정

ID	원문의 EVENT	구현 표식	실제 처리·차이
CR2-02	SELECT_CSV_WEEKLY_HISTORY	427 · 437	weekly와 최근 7일 날짜 범위로 변경
CR2-03	SELECT_CSV_MONTHLY_HISTORY	427 · 442	monthly와 최근 30일 날짜 범위로 변경
CR2-04	SELECT_CSV_DATE	427 · 450	custom.idle 진입; 날짜·달력 입력 활성화
CR2-05	SELECT_CSV_TODAY_HISTORY	437	today와 오늘 날짜 범위로 변경
CR2-06	SELECT_CSV_MONTHLY_HISTORY	437 · 442	monthly와 최근 30일 날짜 범위로 변경
CR2-07	SELECT_CSV_DATE	437 · 450	custom.idle 진입; 날짜·달력 입력 활성화
CR2-08	SELECT_CSV_TODAY_HISTORY	442	today와 오늘 날짜 범위로 변경
CR2-09	SELECT_CSV_WEEKLY_HISTORY	442	weekly와 최근 7일 날짜 범위로 변경
CR2-10	SELECT_CSV_DATE	442 · 450	custom.idle 진입; 날짜·달력 입력 활성화
CR2-11	SELECT_CSV_TODAY_HISTORY	450	today와 오늘 날짜 범위로 변경
CR2-12	SELECT_CSV_WEEKLY_HISTORY	450	weekly와 최근 7일 날짜 범위로 변경
CR2-13	SELECT_CSV_MONTHLY_HISTORY	450	monthly와 최근 30일 날짜 범위로 변경
CR2-14	START_CSV_START_DATE_SELECTION	450 · 466	start_calendar 진입·시작 달력 표시
CR2-15	START_CSV_FINISH_DATE_SELECTION	450 · 484	end_calendar 진입·종료 달력 표시
CR2-16	START_DATE_SELECTED	466	is_start_date_valid 통과 → store_start_date, 달력 유지
CR2-17	START_DATE_SELECTED	466	시작일 guard 실패 → remember_invalid_start_date
CR2-18	START_DATE_CALENDAR_OUTSIDE_CLICKED	466	custom.idle 복귀·달력 닫기
CR2-19	FINISH_DATE_SELECTED	484	is_end_date_valid 통과 → store_end_date, 달력 유지

ID	원문의 EVENT	구현 표식	실제 처리·차이
CR2-20	FINISH_DATE_SELECTED	484	종료일 guard 실패 → remember_invalid_end_date
CR2-21	FINISH_DATE_CALENDAR_OUTSIDE_CLICKED	484	custom.idle 복귀·달력 닫기

CR3 — 파일명

실행 정의: [csvExportMachine.ts](#)

ID	원문의 EVENT	구현 표식	실제 처리·차이
CR3-01	None	511	committed와 기본 파일명 context
CR3-02	FILE_NAME_CLICKED	511 · 517	editing으로 전환
CR3-03	ENTER_KEY_TYPED	517	유효한 Enter → commit_file_name·committed
CR3-04	FILE_NAME_INPUT_FOCUS_LOST	517	유효한 포커스 해제 → commit_file_name·committed
CR3-05	ENTER_KEY_TYPED	517	잘못된 Enter → 오류·editing 유지
CR3-06	FILE_NAME_INPUT_FOCUS_LOST	517	잘못된 포커스 해제 → 오류·editing 유지
CR3-07	FILE_NAME_CLICKED	511	committed → editing; 기본값/수정값 상태는 합쳐 표현

ES3 — 프로그램 종료

실행 정의: [appExitMachine.ts](#)

ID	원문의 EVENT	구현 표식	실제 처리·차이
ES3-01	None	239	awaiting_exit
ES3-02	EXIT_CLICKED	239 · 255	포지션 있음 → force_sell_exit_confirmation
ES3-03	EXIT_CLICKED	239 · 328	포지션 없음 → exit_confirmation
ES3-04	FORCE_SELL_EXIT_CANCELED	255	확인 취소 → awaiting_exit
ES3-05	FORCE_SELL_EXIT_CONFIRMED	255 · 267	force_selling invoke; 복구 포지션은 별도 청산 명령

ID	원문의 EVENT	구현 표식	실제 처리·차이
ES3-06	FORCE_SELL_EXIT_SUCCEEDED	267 · 340 · 380	terminal 확인 → shutdown invoke → final → 실제 창 종료
ES3-07	FORCE_SELL_EXIT_FAILED	255 · 267	onError 또는 결과 이상 → 오류·청산 확인창 복귀
ES3-08	EXIT_CANCELED	328	일반 종료 취소 → awaiting_exit
ES3-09	EXIT_CONFIRMED	328 · 340 · 380	shutting_down invoke 완료 후 final; 실패·복구 분기 존재